

Gas Report

Before:

Solc version: 0.8.20		Optimizer enabled: false		Runs: 200	Block limit: 60000000 gas	
Methods						
Contract	Method	Min	Max	Avg	# calls	usd (avg)
Auction	createAuction	126021	143121	142102	17	-
Auction	endAuction	57122	81853	63305	4	-
Auction	placeBid	67735	81465	76472	11	-
Auction	withdrawBid	-	-	31705	2	-
ReentrancyAttacker	attackWithdraw	-	-	92710	1	-
ReentrancyAttacker	placeAttackBid	-	-	93744	1	-
Deployments					% of limit	
Auction		-	-	1814010	3 %	-
ReentrancyAttacker		-	-	460617	0.8 %	-

After:

Solc version: 0.8.20		Optimizer enabled: false		Runs: 200	Block limit: 60000000 gas	
Methods						
Contract	Method	Min	Max	Avg	# calls	usd (avg)
Auction	createAuction	100472	117572	116609	18	-
Auction	endAuction	36527	61350	42733	4	-
Auction	placeBid	56136	61556	58107	11	-
Auction	withdrawBid	-	-	31705	2	-
ReentrancyAttacker	attackWithdraw	-	-	92710	1	-
ReentrancyAttacker	placeAttackBid	-	-	68415	1	-
Deployments					% of limit	
Auction		-	-	2015747	3.4 %	-
ReentrancyAttacker		-	-	460617	0.8 %	-

Gas Optimization Analysis

This report compares gas consumption before and after optimization of the smart contract.

Function	Metric	Before	After	Difference
createAuction	Min	126021	108472	-17549
createAuction	Max	143121	117572	-25549
createAuction	Avg	142102	116609	-25493
endAuction	Min	57122	36527	-20595
endAuction	Max	81853	61350	-20503
endAuction	Avg	63305	42733	-20572
placeBid	Min	67735	56136	-11599
placeBid	Max	81465	61556	-19909
placeBid	Avg	76472	58107	-18365
withdrawBid	Avg	31705	31705	0
attackWithdraw	Avg	92710	92710	0
placeAttackBid	Avg	93744	68415	-25329

Contract	Before	After	Difference
Auction	1814010	2015747	+201737
ReentrancyAttacker	466617	466617	0

1. Key Optimizations Applied

- **Struct Packing (MAJOR Improvement):**

Before:

```
uint256 highestBid;  
uint256 startingPrice;  
uint256 deadline;  
bool ended;
```

After:

```
uint96 highestBid;  
uint96 startingPrice;  
uint64 deadline;  
bool ended;
```

Why this improves gas:

- Solidity stores variables in **32-byte** storage slots

- Smaller data types allow multiple variables to fit in one slot (**storage packing**)
- This reduces:
 - Storage writes (SSTORE)
 - Storage reads (SLOAD)

Result: Significant gas reduction during createAuction() and updates

2. Use of calldata instead of memory

Applied in:

```
function createAuction(
    string calldata _itemName,
    string calldata _metadataCID
```

Why this improves gas:

- calldata is read-only and cheaper
- Avoids unnecessary memory allocation

Result: Lower gas cost for external function calls

3. Caching Storage Variables

Example:

```
AuctionItem storage a = auctions[_auctionId];
```

Why this improves gas:

- Avoids repeated access like:

```
auctions[_auctionId ]. highestBid
```

- Each storage read is expensive

Result: Reduces repeated **SLOAD** operations

4. Pull Over Push Payment Pattern

Instead of:

- Directly sending ETH inside loops or functions

Used:

```
pendingReturns[_auctionId][prevBidder] += prevBid;
```

Why this improves gas:

- Avoids failed transactions due to transfer issues

- Reduces unnecessary external calls

Result:

- More efficient and safer execution

5. Increment Optimization

Before:

```
auctionCount++;
```

After:

```
uint256 id = ++auctionCount;
```

Why this improves gas:

- Slightly cheaper due to pre-increment usage
- Avoids extra temporary operations

Minor but measurable improvement

6. Reduced Redundant Storage Access

Using:

```
address prevBidder = a.highestBidder;
```

```
uint256 prevBid = a.highestBid;
```

Why this improves gas:

- Prevents multiple reads from storage
- Uses stack variables instead

Summary:

Gas optimisation was applied to reduce the runtime cost of core functions in the smart contract. Key improvements include struct packing using smaller data types (**uint96**, **uint64**), use of **calldata** instead of memory, and caching storage variables to minimize repeated storage access.

These changes significantly reduced gas consumption during function execution such as **createAuction** and **placeBid**. Although deployment gas slightly increased due to added structure and validations, the overall contract became more efficient for repeated use.